

ASSIGNMENT 6 ADDRESS RESOLUTION PROTOCOL (ARP)

Name: Nivetha Dhanakoti
Reg no.: 3122 22 5001 087

Aim: To Simulate ARP using socket programming.

Server should perform the following:

1. Consider the server as a host or a router.
2. Enter hosts/routers' IP address and MAC address.
3. Listen for any number of client (for broadcasting purpose)
4. Enter the packet details received from a host or its own packet to sent to a destination.

The details are:

- i. Source IP address
- ii. Source MAC address
- iii. Destination IP address
- iv. 16 bit data

5. Develop an ARP Request packet which is to be broadcasted to all clients. Query packet should contain SourceIP | SourceMAC | DestinationIP
6. When an ARP Reply is received with the Destination MAC address, send the packet to the corresponding destination with format SourceIP | SourceMAC | DestinationIP|Destination MAC Address|16 bit data
7. Also check the validity of IP and MAC address.

Client should do the following:

1. Can have any number of clients(depends on the backlog). A single program is enough for any number of clients.
2. Enter the clients own IP and MAC.
3. When an ARP Request packet is received check whether Destination IP is its own IP.
4. If not no reply.
5. If yes respond with ARP Reply packet. SourceIP | SourceMAC | DestinationIP
DestinationMAC
6. Then receive the packet from the server and display it.

Sample Input and Output

Server

Enter the details of packet received.

Destination IP :155.157.65.128

Source IP :123.128.34.56

Source MAC :AF-45-E5-00-97-12

16 bit data :1011110000101010

Developing ARP Request packet 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128

The ARP Request packet is broadcasted.

Waiting for ARP Reply...

ARP Reply received

123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2

Sending the packet to : 45-DA-62-21-1A-B2

Packet Sent: 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2 |

011110000101010

Client 1

Enter the IP address : 165.43.158.158

Enter the Mac address : 09-DF-90-26-6C-09

ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128

IP address does not match.

Client 2

Enter the IP address : 155.157.65.128

Enter the Mac address : 45-DA-62-21-1A-B2

ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128

IP address matches

ARP Reply Sent : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2

Received Packet is : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128 | 45-DA-62-21-1A-B2 | 011110000101010

Client 3

Enter the IP address : 15.143.158.18

Enter the Mac address : 19-0F-01-63-C7-D4

ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128

IP address does not match

Server Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <netinet/in.h>
#define BUFFER_SIZE 1024
#define PORT 42345

// Function to check if IP address is valid
int is_valid_ip(const char *ip) {
    struct sockaddr_in sa;
    return inet_pton(AF_INET, ip, &(sa.sin_addr)) != 0;
}

// Function to check if MAC address is valid
int is_valid_mac(const char *mac) {
    int i;
    if (strlen(mac) != 17) return 0;
    for (i = 0; i < 17; i++) {
        if ((i + 1) % 3 == 0) {
            if (mac[i] != '-') return 0;
        } else {
            if (!((mac[i] >= '0' && mac[i] <= '9') || (mac[i] >= 'A' && mac[i] <= 'F') || (mac[i] >= 'a' && mac[i] <= 'f'))) return 0;
        }
    }
    return 1;
}

void handle_client(int sockfd) {
    struct sockaddr_in client_addr;
    socklen_t addr_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE];
    char src_ip[16], src_mac[18], dest_ip[16], dest_mac[18];
    uint16_t data;
    ssize_t recv_len;

    while (1) {
        // Receive ARP Request
```

```

    recv_len = recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr *)&client_addr,
&addr_len);
    if (recv_len < 0) {
        perror("recvfrom failed");
        exit(EXIT_FAILURE);
    }

    // Parse packet
    sscanf(buffer, "%15s %17s %15s %5hx", src_ip, src_mac, dest_ip, &data);

    printf("Received ARP Request:\n");
    printf("Source IP: %s\n", src_ip);
    printf("Source MAC: %s\n", src_mac);
    printf("Destination IP: %s\n", dest_ip);

    // Simulate ARP Request Handling
    if (strcmp(dest_ip, "155.157.65.128") == 0) {
        strcpy(dest_mac, "45-DA-62-21-1A-B2"); // Example MAC address

        // Prepare ARP Reply
        snprintf(buffer, sizeof(buffer), "%s %s %s %s %04X", src_ip, src_mac, dest_ip,
dest_mac, data);
        sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&client_addr, addr_len);

        printf("ARP Reply Sent:\n");
        printf("Source IP: %s\n", src_ip);
        printf("Source MAC: %s\n", src_mac);
        printf("Destination IP: %s\n", dest_ip);
        printf("Destination MAC: %s\n", dest_mac);
        printf("Data: %04X\n", data);

        // Print and send the final packet to destination MAC
        snprintf(buffer, sizeof(buffer), "%s %s %s %s %04X", src_ip, src_mac, dest_ip,
dest_mac, data);
        printf("Packet Sent: %s\n", buffer);
    } else {
        printf("Destination IP does not match.\n");
    }

    // Ask the user if they want to end the server
    char choice;
    printf("Do you want to end the server? (y/n): ");
    scanf(" %c", &choice);
    if (choice == 'y' || choice == 'Y') {
        break;
    }
}
}

int main() {
    int sockfd;
    struct sockaddr_in server_addr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket creation failed");
    }
}

```

```

        exit(EXIT_FAILURE);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("Server is running and waiting for ARP requests...\n");
    handle_client(sockfd);
    close(sockfd);
    return 0;
}

```

Client Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <netinet/in.h>
#include <signal.h>
#define BUFFER_SIZE 1024
#define PORT 42345

// Function to check if IP address is valid
int is_valid_ip(const char *ip) {
    struct sockaddr_in sa;
    return inet_pton(AF_INET, ip, &(sa.sin_addr)) != 0;
}

// Function to check if MAC address is valid
int is_valid_mac(const char *mac) {
    int i;
    if (strlen(mac) != 17) return 0;
    for (i = 0; i < 17; i++) {
        if ((i + 1) % 3 == 0) {
            if (mac[i] != '-') return 0;
        } else {
            if (!((mac[i] >= '0' && mac[i] <= '9') || (mac[i] >= 'A' && mac[i] <= 'F') || (mac[i] >= 'a' && mac[i] <= 'f')))) return 0;
        }
    }
    return 1;
}

// Signal handler for SIGINT
void signal_handler(int sig) {
    if (sig == SIGINT) {

```

```

        printf("\nClient terminated.\n");
        exit(0);
    }
}

void client_program() {
    int sockfd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE];
    char client_ip[16], client_mac[18];
    char src_ip[16], src_mac[18], dest_ip[16], dest_mac[18];
    uint16_t data;
    socklen_t addr_len = sizeof(server_addr);

    // Set up signal handling
    signal(SIGINT, signal_handler);

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1"); // Assuming server is running locally
    server_addr.sin_port = htons(PORT);

    printf("Enter your IP address: ");
    scanf("%15s", client_ip);
    if (!is_valid_ip(client_ip)) {
        printf("Invalid IP address.\n");
        close(sockfd);
        return;
    }

    printf("Enter your MAC address: ");
    scanf("%17s", client_mac);
    if (!is_valid_mac(client_mac)) {
        printf("Invalid MAC address.\n");
        close(sockfd);
        return;
    }

    // Prepare and send ARP Request
    snprintf(buffer, sizeof(buffer), "%s %s %s %04X", client_ip, client_mac, "155.157.65.128",
0xABCD);
    if (sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&server_addr, addr_len) < 0)
    {
        perror("sendto failed");
        close(sockfd);
        return;
    }
    printf("ARP Request Sent.\n");

    // Receive ARP Reply

```

```

    ssize_t recv_len = recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct sockaddr
*)&server_addr, &addr_len);
    if (recv_len < 0) {
        perror("recvfrom failed");
        close(sockfd);
        return;
    }

    buffer[recv_len] = '\0'; // Null-terminate the received data
    sscanf(buffer, "%15s %17s %15s %17s %5hx", src_ip, src_mac, dest_ip, dest_mac, &data);

    printf("ARP Request Received:\n");
    printf("Source IP: %s\n", src_ip);
    printf("Source MAC: %s\n", src_mac);
    printf("Destination IP: %s\n", dest_ip);

    if (strcmp(dest_ip, client_ip) == 0) {
        // Reply with ARP Response
        snprintf(buffer, sizeof(buffer), "%s %s %s %s %04X", src_ip, src_mac, dest_ip,
client_mac, data);
        if (sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&server_addr, addr_len) <
0) {
            perror("sendto failed");
            close(sockfd);
            return;
        }

        printf("ARP Reply Sent:\n");
        printf("Source IP: %s\n", src_ip);
        printf("Source MAC: %s\n", src_mac);
        printf("Destination IP: %s\n", dest_ip);
        printf("Destination MAC: %s\n", client_mac);
        printf("Data: %04X\n", data);

        // Receive packet from server
        recv_len = recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct sockaddr
*)&server_addr, &addr_len);
        if (recv_len < 0) {
            perror("recvfrom failed");
            close(sockfd);
            return;
        }

        buffer[recv_len] = '\0'; // Null-terminate the received data
        sscanf(buffer, "%15s %17s %15s %17s %5hx", src_ip, src_mac, dest_ip, dest_mac,
&data);

        printf("Received Packet:\n");
        printf("Source IP: %s\n", src_ip);
        printf("Source MAC: %s\n", src_mac);
        printf("Destination IP: %s\n", dest_ip);
        printf("Destination MAC: %s\n", dest_mac);
        printf("Data: %04X\n", data);
    } else {
        printf("IP address does not match.\n");
    }
}

```

```

    }
    close(sockfd);
}

int main() {
    client_program();
    return 0;
}

```

Server Output:

```

nivethadhanakoti@Nivethas-MacBook-Pro A6 % gcc server.c -o server
nivethadhanakoti@Nivethas-MacBook-Pro A6 % ./server
Server is running and waiting for ARP requests...
Received ARP Request:
Source IP: 165.43.158.158
Source MAC: 09-DF-90-26-6C-09
Destination IP: 155.157.65.128
ARP Reply Sent:
Source IP: 165.43.158.158
Source MAC: 09-DF-90-26-6C-09
Destination IP: 155.157.65.128
Destination MAC: 45-DA-62-21-1A-B2
Data: ABCD
Packet Sent: 165.43.158.158 09-DF-90-26-6C-09 155.157.65.128 45-DA-62-21-1A-B2 ABCD
Do you want to end the server? (y/n): n
Received ARP Request:
Source IP: 155.157.65.128
Source MAC: 45-DA-62-21-1A-B2
Destination IP: 155.157.65.128
ARP Reply Sent:
Source IP: 155.157.65.128
Source MAC: 45-DA-62-21-1A-B2
Destination IP: 155.157.65.128
Destination MAC: 45-DA-62-21-1A-B2
Data: BCDA
Packet Sent: 155.157.65.128 45-DA-62-21-1A-B2 155.157.65.128 45-DA-62-21-1A-B2 BCDA
Do you want to end the server? (y/n): y
nivethadhanakoti@Nivethas-MacBook-Pro A6 % █

```

Client Output:

```

nivethadhanakoti@Nivethas-MacBook-Pro A6 % gcc client.c -o client
nivethadhanakoti@Nivethas-MacBook-Pro A6 % ./client
Enter your IP address: 165.43.158.158
Enter your MAC address: 09-DF-90-26-6C-09
ARP Request Sent.
ARP Request Received:
Source IP: 165.43.158.158
Source MAC: 09-DF-90-26-6C-09
Destination IP: 155.157.65.128
IP address does not match.
nivethadhanakoti@Nivethas-MacBook-Pro A6 % gcc client.c -o client2
nivethadhanakoti@Nivethas-MacBook-Pro A6 % ./client2
Enter your IP address: 155.157.65.128
Enter your MAC address: 45-DA-62-21-1A-B2
ARP Request Sent.
ARP Request Received:
Source IP: 155.157.65.128
Source MAC: 45-DA-62-21-1A-B2
Destination IP: 155.157.65.128
ARP Reply Sent:
Source IP: 155.157.65.128
Source MAC: 45-DA-62-21-1A-B2
Destination IP: 155.157.65.128
Destination MAC: 45-DA-62-21-1A-B2
Data: BCDA

```

Learning Outcomes:

I have learnt to simulate ARP using socket programming.